

feature-selection-using-correlation

August 12, 2025

1 Filter Method

```
[18]: import pandas as pd

# Load the dataset
df = pd.read_csv('feature_selection.csv')
```

```
[19]: df
```

```
[19]:  location    area  bedroom  bathroom owner  blodd group  price
0         a    dhaka        3         2     x      o+  10000
1         b    dhaka        4         2     y      ab+  12000
2         c  khulna        2         2     p      ab-  11000
3         d rajshahi        3         2     q      o+  15000
4         e  cumilla        3         1     w      ab+  12000
5         f    dhaka        2         1     l      ab-  11000
6         g  barishal        2         1     r      o+  10000
7         h   natore        2         3     s      o+  15000
8         i  khulna        3         3     k      ab+  12000
9         j    dhaka        4         2     v      ab-  11000
```

```
[20]: # Check for non-numeric data in the columns
print(df.dtypes)
```

```
location    object
area        object
bedroom     int64
bathroom    int64
owner       object
blodd group object
price       int64
dtype: object
```

```
[21]: # Identify non-numeric values in specific columns
for column in df.columns:
    if df[column].dtype == 'object':
        print(f"Non-numeric values in '{column}':\n", df[column].unique())
```

```

Non-numeric values in 'location ':
['a' 'b' 'c' 'd' 'e' 'f' 'g' 'h' 'i' 'j']
Non-numeric values in 'area ':
['dhaka' 'khulna' 'rajshahi' 'cumilla' 'barishal' 'natore']
Non-numeric values in 'owner ':
['x' 'y' 'p' 'q' 'w' 'l' 'r' 's' 'k' 'v']
Non-numeric values in 'blodd group':
['o+' 'ab+' 'ab-']

```

```

[22]: # Option 1: Convert columns to numeric, forcing errors to NaN
df = df.apply(pd.to_numeric, errors='coerce')

```

```

[23]: df

```

```

[23]:   location  area  bedroom  bathroom  owner  blodd group  price
0      NaN    NaN        3          2    NaN      NaN    10000
1      NaN    NaN        4          2    NaN      NaN    12000
2      NaN    NaN        2          2    NaN      NaN    11000
3      NaN    NaN        3          2    NaN      NaN    15000
4      NaN    NaN        3          1    NaN      NaN    12000
5      NaN    NaN        2          1    NaN      NaN    11000
6      NaN    NaN        2          1    NaN      NaN    10000
7      NaN    NaN        2          3    NaN      NaN    15000
8      NaN    NaN        3          3    NaN      NaN    12000
9      NaN    NaN        4          2    NaN      NaN    11000

```

```

[ ]: # Option 2: Alternatively, remove or replace non-numeric values
# Replace non-numeric values with NaN
df.replace(['a', 'N/A', 'unknown'], float('nan'), inplace=True)

# Drop rows with NaN values (if appropriate)
df.dropna(inplace=True)

```

```

[10]: # Calculate correlation with the target variable 'price'
correlation = df.corr()

```

```

-----
ValueError                                Traceback (most recent call last)
Cell In[10], line 2
      1 # Calculate correlation with the target variable 'price'
----> 2 correlation = df.corr()

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\frame.py:10054, in_
DataFrame.corr(self, method, min_periods, numeric_only)
    10052 cols = data.columns
    10053 idx = cols.copy()
> 10054 mat = data.to_numpy(dtype=float, na_value=np.nan, copy=False)

```

```

10056 if method == "pearson":
10057     correl = libalgos.nancorr(mat, minp=min_periods)

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\frame.py:1838, in DataFrame.to_numpy(self, dtype, copy, na_value)
    1836 if dtype is not None:
    1837     dtype = np.dtype(dtype)
-> 1838 result = self._mgr.as_array(dtype=dtype, copy=copy, na_value=na_value)
    1839 if result.dtype is not dtype:
    1840     result = np.array(result, dtype=dtype, copy=False)

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\internals\managers.py:1732, in BlockManager.as_array(self, dtype, copy, na_value)
    1730     arr.flags.writeable = False
    1731 else:
-> 1732     arr = self._interleave(dtype=dtype, na_value=na_value)
    1733     # The underlying data was copied within _interleave, so no need
    1734     # to further copy if copy=True or setting na_value
    1736 if na_value is not lib.no_default:

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\internals\managers.py:1794, in BlockManager._interleave(self, dtype, na_value)
    1792     else:
    1793         arr = blk.get_values(dtype)
-> 1794     result[rl.indexer] = arr
    1795     itemmask[rl.indexer] = 1
    1797 if not itemmask.all():

ValueError: could not convert string to float: 'a'

```

```

[ ]: # Display the correlation of each feature with the target variable 'price'
correlated_features = correlation['price'].sort_values(ascending=False)
print("Correlation with Price:\n", correlated_features)

```

```

[6]: print(df.dtypes)

```

```

location      object
area          object
bedroom       int64
bathroom      int64
owner         object
blodd group   object
dtype: object

```

```

[14]: # Option 1: Convert columns to numeric, forcing errors to NaN
df = df.apply(pd.to_numeric, errors='coerce')

```

```
[15]: # Calculate correlation with the target variable 'price'
correlation = df.corr()
```

```
[16]: # Display the correlation of each feature with the target variable 'price'
correlated_features = correlation['price'].sort_values(ascending=False)
print("Correlation with Price:\n", correlated_features)
```

```
Correlation with Price:
price          1.000000
bathroom      0.495798
bedroom       -0.015721
location      NaN
area          NaN
owner         NaN
blodd group   NaN
Name: price, dtype: float64
```

```
[17]: df
```

```
[17]:
```

	location	area	bedroom	bathroom	owner	blodd group	price
0	NaN	NaN	3	2	NaN	NaN	10000
1	NaN	NaN	4	2	NaN	NaN	12000
2	NaN	NaN	2	2	NaN	NaN	11000
3	NaN	NaN	3	2	NaN	NaN	15000
4	NaN	NaN	3	1	NaN	NaN	12000
5	NaN	NaN	2	1	NaN	NaN	11000
6	NaN	NaN	2	1	NaN	NaN	10000
7	NaN	NaN	2	3	NaN	NaN	15000
8	NaN	NaN	3	3	NaN	NaN	12000
9	NaN	NaN	4	2	NaN	NaN	11000

```
[24]: from sklearn.preprocessing import OrdinalEncoder
```

```
[26]: categorical_column = 'location'
```

```
[27]: encoder = OrdinalEncoder()
df[categorical_column] = encoder.fit_transform(df[[categorical_column]])
```

```
-----
KeyError                                Traceback (most recent call last)
Cell In[27], line 2
      1 encoder = OrdinalEncoder()
----> 2 df[categorical_column] = encoder.fit_transform(df[[categorical_column]])

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\frame.py:3767, in DataFrame._getitem__(self, key)
    3765     if is_iterator(key):
```

```

3766         key = list(key)
-> 3767         indexer = self.columns._get_indexer_strict(key, "columns")[1]
3769 # take() does not accept boolean indexers
3770 if getattr(indexer, "dtype", None) == bool:

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:
  5877, in Index._get_indexer_strict(self, key, axis_name)
    5874 else:
    5875     keyarr, indexer, new_indexer = self._reindex_non_unique(keyarr)
-> 5877 self._raise_if_missing(keyarr, indexer, axis_name)
    5879 keyarr = self.take(indexer)
    5880 if isinstance(key, Index):
    5881     # GH 42790 - Preserve name from an Index

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:
  5938, in Index._raise_if_missing(self, key, indexer, axis_name)
    5936     if use_interval_msg:
    5937         key = list(key)
-> 5938     raise KeyError(f"None of [{key}] are in the [{axis_name}]")
    5940 not_found = list(ensure_index(key)[missing_mask.nonzero()[0]].unique())
    5941 raise KeyError(f"{not_found} not in index")

KeyError: "None of [Index(['location'], dtype='object')] are in the [columns]"

```

2 Start from here

```

[28]: import pandas as pd

# Load the dataset
df = pd.read_csv('feature_selection.csv')

```

```

[29]: df

```

```

[29]:   location  area  bedroom  bathroom  owner  blodd  group  price
0         a  dhaka         3         2     x      o+  10000
1         b  dhaka         4         2     y      ab+  12000
2         c  khulna        2         2     p      ab-  11000
3         d rajshahi        3         2     q      o+  15000
4         e  cumilla        3         1     w      ab+  12000
5         f  dhaka         2         1     l      ab-  11000
6         g barishal        2         1     r      o+  10000
7         h  natore        2         3     s      o+  15000
8         i  khulna        3         3     k      ab+  12000
9         j  dhaka         4         2     v      ab-  11000

```

```
[30]: # Check for non-numeric data in the columns
print(df.dtypes)
```

```
location      object
area          object
bedroom       int64
bathroom      int64
owner         object
blodd group   object
price         int64
dtype: object
```

```
[31]: # Identify non-numeric values in specific columns
for column in df.columns:
    if df[column].dtype == 'object':
        print(f"Non-numeric values in '{column}':\n", df[column].unique())
```

```
Non-numeric values in 'location ':
['a' 'b' 'c' 'd' 'e' 'f' 'g' 'h' 'i' 'j']
Non-numeric values in 'area ':
['dhaka' 'khulna' 'rajshahi' 'cumilla' 'barishal' 'natore']
Non-numeric values in 'owner ':
['x' 'y' 'p' 'q' 'w' 'l' 'r' 's' 'k' 'v']
Non-numeric values in 'blodd group':
['o+' 'ab+' 'ab-']
```

3 Encoding / Handling another column => location

```
[32]: from sklearn.preprocessing import OrdinalEncoder
```

```
[36]: categorical_column = 'location'
```

```
[35]: df
```

```
[35]:
```

	location	area	bedroom	bathroom	owner	blodd group	price
0	a	dhaka	3	2	x	o+	10000
1	b	dhaka	4	2	y	ab+	12000
2	c	khulna	2	2	p	ab-	11000
3	d	rajshahi	3	2	q	o+	15000
4	e	cumilla	3	1	w	ab+	12000
5	f	dhaka	2	1	l	ab-	11000
6	g	barishal	2	1	r	o+	10000
7	h	natore	2	3	s	o+	15000
8	i	khulna	3	3	k	ab+	12000
9	j	dhaka	4	2	v	ab-	11000

```
[34]: encoder = OrdinalEncoder()
df[categorical_column] = encoder.fit_transform(df[[categorical_column]])
```

```
-----
KeyError                                Traceback (most recent call last)
Cell In[34], line 2
      1 encoder = OrdinalEncoder()
----> 2 df[categorical_column] = encoder.fit_transform(df[[categorical_column]])

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\frame.py:3767, in DataFrame._getitem__(self, key)
    3765     if is_iterator(key):
    3766         key = list(key)
-> 3767     indexer = self.columns._get_indexer_strict(key, "columns")[1]
    3769 # take() does not accept boolean indexers
    3770 if getattr(indexer, "dtype", None) == bool:

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:
5877, in Index._get_indexer_strict(self, key, axis_name)
    5874 else:
    5875     keyarr, indexer, new_indexer = self._reindex_non_unique(keyarr)
-> 5877 self._raise_if_missing(keyarr, indexer, axis_name)
    5879 keyarr = self.take(indexer)
    5880 if isinstance(key, Index):
    5881     # GH 42790 - Preserve name from an Index

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:
5938, in Index._raise_if_missing(self, key, indexer, axis_name)
    5936     if use_interval_msg:
    5937         key = list(key)
-> 5938     raise KeyError(f"None of [{key}] are in the [{axis_name}]")
    5940 not_found = list(ensure_index(key)[missing_mask.nonzero()[0]].unique())
    5941 raise KeyError(f"{not_found} not in index")

KeyError: "None of [Index(['location'], dtype='object')] are in the [columns]"
```

```
[37]: encoder = OrdinalEncoder()
df[categorical_column] = encoder.fit_transform(df[[categorical_column]])
```

```
-----
KeyError                                Traceback (most recent call last)
Cell In[37], line 2
      1 encoder = OrdinalEncoder()
----> 2 df[categorical_column] = encoder.fit_transform(df[[categorical_column]])
```

```

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\frame.py:3767, in
↳ DataFrame.__getitem__(self, key)
    3765     if is_iterator(key):
    3766         key = list(key)
-> 3767     indexer = self.columns._get_indexer_strict(key, "columns")[1]
    3769 # take() does not accept boolean indexers
    3770 if getattr(indexer, "dtype", None) == bool:

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:
↳ 5877, in Index._get_indexer_strict(self, key, axis_name)
    5874 else:
    5875     keyarr, indexer, new_indexer = self._reindex_non_unique(keyarr)
-> 5877 self._raise_if_missing(keyarr, indexer, axis_name)
    5879 keyarr = self.take(indexer)
    5880 if isinstance(key, Index):
    5881     # GH 42790 - Preserve name from an Index

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:
↳ 5938, in Index._raise_if_missing(self, key, indexer, axis_name)
    5936     if use_interval_msg:
    5937         key = list(key)
-> 5938     raise KeyError(f"None of [{key}] are in the [{axis_name}]")
    5940 not_found = list(ensure_index(key)[missing_mask.nonzero()[0]].unique())
    5941 raise KeyError(f"{not_found} not in index")

KeyError: "None of [Index(['location'], dtype='object')] are in the [columns]"

```

```
[38]: print(df.columns)
```

```

Index(['location ', 'area ', 'bedroom ', 'bathroom', 'owner ', 'blodd group',
      'price'],
      dtype='object')

```

```

[39]: df.columns = df.columns.str.strip() # Remove any leading/trailing spaces
df.columns = df.columns.str.lower() # Convert all column names to lowercase
↳ (optional)
print(df.columns)

```

```

Index(['location', 'area', 'bedroom', 'bathroom', 'owner', 'blodd group',
      'price'],
      dtype='object')

```

```

[40]: # Optional: Use a conditional check
if 'location' in df.columns:
    print(df['location'].head())
else:
    print("The 'location' column is not in the DataFrame.")

```

```

0    a
1    b
2    c
3    d
4    e
Name: location, dtype: object

```

```
[41]: encoder = OrdinalEncoder()
df[categorical_column] = encoder.fit_transform(df[[categorical_column]])
```

```
[42]: df
```

```
[42]:
```

	location	area	bedroom	bathroom	owner	blodd	group	price
0	0.0	dhaka	3	2	x		o+	10000
1	1.0	dhaka	4	2	y		ab+	12000
2	2.0	khulna	2	2	p		ab-	11000
3	3.0	rajshahi	3	2	q		o+	15000
4	4.0	cumilla	3	1	w		ab+	12000
5	5.0	dhaka	2	1	l		ab-	11000
6	6.0	barishal	2	1	r		o+	10000
7	7.0	natore	2	3	s		o+	15000
8	8.0	khulna	3	3	k		ab+	12000
9	9.0	dhaka	4	2	v		ab-	11000

4 Encoding / Handling another column => area

```
[43]: categorical_column = 'area'
```

```
[44]: encoder = OrdinalEncoder()
df[categorical_column] = encoder.fit_transform(df[[categorical_column]])
```

```
[45]: df
```

```
[45]:
```

	location	area	bedroom	bathroom	owner	blodd	group	price
0	0.0	2.0	3	2	x		o+	10000
1	1.0	2.0	4	2	y		ab+	12000
2	2.0	3.0	2	2	p		ab-	11000
3	3.0	5.0	3	2	q		o+	15000
4	4.0	1.0	3	1	w		ab+	12000
5	5.0	2.0	2	1	l		ab-	11000
6	6.0	0.0	2	1	r		o+	10000
7	7.0	4.0	2	3	s		o+	15000
8	8.0	3.0	3	3	k		ab+	12000
9	9.0	2.0	4	2	v		ab-	11000

5 Encoding more than one column at a time

```
[46]: categorical_columns = ['owner', 'blodd group'] # Add or replace with your
      ↪desired column names
```

```
[47]: for column in categorical_columns:
      if column not in df.columns:
          print(f"Column '{column}' does not exist in the DataFrame")
      else:
          print(f"Encoding column: {column}")
```

Encoding column: owner

Encoding column: blodd group

```
[48]: encoder = OrdinalEncoder()
      existing_categorical_columns = [col for col in categorical_columns if col in df.
      ↪columns]
      df[existing_categorical_columns] = encoder.
      ↪fit_transform(df[existing_categorical_columns])
```

```
[49]: df
```

```
[49]:
```

	location	area	bedroom	bathroom	owner	blodd group	price
0	0.0	2.0	3	2	8.0	2.0	10000
1	1.0	2.0	4	2	9.0	0.0	12000
2	2.0	3.0	2	2	2.0	1.0	11000
3	3.0	5.0	3	2	3.0	2.0	15000
4	4.0	1.0	3	1	7.0	0.0	12000
5	5.0	2.0	2	1	1.0	1.0	11000
6	6.0	0.0	2	1	4.0	2.0	10000
7	7.0	4.0	2	3	5.0	2.0	15000
8	8.0	3.0	3	3	0.0	0.0	12000
9	9.0	2.0	4	2	6.0	1.0	11000

```
[50]: # Calculate correlation with the target variable 'price'
      correlation = df.corr()
```

```
[51]: # Display the correlation of each feature with the target variable 'price'
      correlated_features = correlation['price'].sort_values(ascending=False)
      print("Correlation with Price:\n", correlated_features)
```

Correlation with Price:

price	1.000000
area	0.797921
bathroom	0.495798
blodd group	0.148712
location	0.133118

```
bedroom      -0.015721
owner        -0.092159
Name: price, dtype: float64
```